

I. DERIVATIVES ON THE CONSTRAINT MANIFOLD IN GENERICJOINT

A. Constraint Structure and Coordinate Partitioning

The `Generic` cluster joint supports implicit loop constraints of the form $\phi(\mathbf{q}_{\text{sp}}) = \mathbf{0}$, where $\mathbf{q}_{\text{sp}} \in \mathbb{R}^{n_{\text{sp}}}$ are the spanning coordinates of the cluster and $\phi : \mathbb{R}^{n_{\text{sp}}} \rightarrow \mathbb{R}^{n_c}$ is a user-supplied smooth constraint function with n_c scalar constraints.

The spanning coordinates are partitioned into n_v *independent* coordinates $\mathbf{y}$ and n_c *dependent* coordinates $\mathbf{q}_{\text{dep}}$, with $n_v + n_c = n_{\text{sp}}$. A permutation matrix $\mathbf{P}$ (stored as `coord_map`) satisfies

$$\mathbf{q}_{\text{sp}} = \mathbf{P} \begin{bmatrix} \mathbf{y} \\ \mathbf{q}_{\text{dep}} \end{bmatrix}, \quad (1)$$

reordering the stacked independent and dependent coordinates into the original spanning order. Without loss of generality, we assume $\mathbf{P}$ is identity in the remainder of the development.

The constraint Jacobian $\mathbf{K} = \partial\phi/\partial\mathbf{q}_{\text{sp}} \in \mathbb{R}^{n_c \times n_{\text{sp}}}$ is partitioned as

$$\mathbf{K} = [\mathbf{K}_i \quad \mathbf{K}_d], \quad (2)$$

where $\mathbf{K}_i \in \mathbb{R}^{n_c \times n_v}$ and $\mathbf{K}_d \in \mathbb{R}^{n_c \times n_c}$ correspond to the independent and dependent coordinate columns respectively. Differentiating $\phi(\mathbf{q}_{\text{sp}}) = \mathbf{0}$ with respect to time and applying (2) gives

$$\mathbf{K}_i \dot{\mathbf{y}} + \mathbf{K}_d \dot{\mathbf{q}}_{\text{dep}} = \mathbf{0}, \quad (3)$$

so that (assuming $\mathbf{K}_d$ is invertible at valid configurations)

$$\dot{\mathbf{q}}_{\text{dep}} = -\mathbf{K}_d^{-1} \mathbf{K}_i \dot{\mathbf{y}}. \quad (4)$$

Stacking (4) with the identity $\dot{\mathbf{y}} = \mathbf{I} \dot{\mathbf{y}}$ and yields the velocity constraint

$$\dot{\mathbf{q}}_{\text{sp}} = \begin{bmatrix} \mathbf{I} \\ -\mathbf{K}_d^{-1} \mathbf{K}_i \end{bmatrix} \dot{\mathbf{y}} \triangleq \mathbf{G}(\mathbf{q}_{\text{sp}}) \dot{\mathbf{y}}, \quad (5)$$

where $\mathbf{G} \in \mathbb{R}^{n_{\text{sp}} \times n_v}$ is the configuration-dependent constraint Jacobian whose columns span $\text{Null}(\mathbf{K})$.

The velocity-level constraint also admits a bias term $\mathbf{g}$ such that $\dot{\mathbf{q}}_{\text{sp}} = \mathbf{G} \dot{\mathbf{y}} + \mathbf{g}$, where

$$\mathbf{g} = \begin{bmatrix} \mathbf{0} \\ \mathbf{K}_d^{-1} \mathbf{k} \end{bmatrix}, \quad \mathbf{k} = -\dot{\mathbf{K}} \dot{\mathbf{q}}_{\text{sp}}. \quad (6)$$

B. Motion Subspace and Bias

The cluster motion subspace $\mathbf{S} \in \mathbb{R}^{6n \times n_v}$, where n is the number of bodies in the cluster, is

$$\mathbf{S}(\mathbf{q}_{\text{sp}}) = \mathbf{X}_{\text{intra}}(\mathbf{q}_{\text{sp}}) \mathbf{S}_{\text{sp}} \mathbf{G}(\mathbf{q}_{\text{sp}}), \quad (7)$$

where $\mathbf{S}_{\text{sp}} \in \mathbb{R}^{6n \times n_{\text{sp}}}$ is a constant block-diagonal matrix with the motion subspace matrix for the spanning tree (one block per spanning tree joint), and $\mathbf{X}_{\text{intra}} \in \mathbb{R}^{6n \times 6n}$ is the lower-block-triangular intra-cluster spatial transform built from the connectivity graph of the spanning tree bodies with blocks given by:

$$[\mathbf{X}_{\text{intra}}]_{ij} = \begin{cases} {}^i\mathbf{X}_j & \text{if body } j \text{ is an ancestor of body } i, \\ \mathbf{0} & \text{otherwise,} \end{cases} \quad (8)$$

where ${}^i\mathbf{X}_j$ is the spatial transform from j to i .

The joint velocity and bias acceleration are

$$\mathbf{v}_J = \mathbf{S} \dot{\mathbf{y}}, \quad (9)$$

$$\mathbf{c}_J = \dot{\mathbf{X}}_{\text{intra}} \mathbf{S}_{\text{sp}} \dot{\mathbf{q}}_{\text{sp}} + \mathbf{X}_{\text{intra}} \mathbf{S}_{\text{sp}} \mathbf{g}, \quad (10)$$

where $\dot{\mathbf{X}}_{\text{intra}}$ captures the velocity-dependent rate of change of the intra-cluster transforms, and $\mathbf{c}_J$ gives the apparent derivative in moving coordinates often denoted $\dot{\mathbf{S}}\dot{\mathbf{y}}$.

C. Derivatives with Respect to Independent Coordinates

The inverse dynamics derivative algorithms require three configuration-dependent Jacobians with respect to the *independent* coordinates $\mathbf{y}$:

$$\frac{\partial(\mathbf{S}\mathbf{b})}{\partial\mathbf{y}}, \quad \frac{\partial(\mathbf{S}^\top\mathbf{F})}{\partial\mathbf{y}}, \quad \frac{\partial\mathbf{c}_J}{\partial\mathbf{y}}, \quad (11)$$

for given fixed vectors $\mathbf{b} \in \mathbb{R}^{n_v}$ and $\mathbf{F} \in \mathbb{R}^{6n}$.

Since $\mathbf{S}$ and $\mathbf{c}_J$ depend on $\mathbf{q}_{\text{sp}}$ rather than $\mathbf{y}$ directly, each quantity is obtained via the chain rule. For any smooth $f(\mathbf{q}_{\text{sp}})$,

$$\frac{\partial f}{\partial\mathbf{y}} = \frac{\partial f}{\partial\mathbf{q}_{\text{sp}}} \mathbf{G}(\mathbf{q}_{\text{sp}}), \quad (12)$$

since $\partial\mathbf{q}_{\text{sp}}/\partial\mathbf{y} = \mathbf{G}$ on the constraint manifold $\mathcal{M} = \{\mathbf{q}_{\text{sp}} \mid \phi(\mathbf{q}_{\text{sp}}) = \mathbf{0}\}$.

Geometrically, multiplying by $\mathbf{G}$ projects the spanning-coordinate Jacobian onto $T_{\mathbf{q}}\mathcal{M} = \text{Null}(\mathbf{K})$, so each column of (12) is the Lie derivative of f along the corresponding tangent direction to $\mathcal{M}$. Applying (12) to each quantity in (11):

$$\frac{\partial(\mathbf{S}\mathbf{b})}{\partial\mathbf{y}} = \frac{\partial(\mathbf{S}\mathbf{b})}{\partial\mathbf{q}_{\text{sp}}} \mathbf{G}, \quad (13)$$

$$\frac{\partial(\mathbf{S}^\top\mathbf{F})}{\partial\mathbf{y}} = \frac{\partial(\mathbf{S}^\top\mathbf{F})}{\partial\mathbf{q}_{\text{sp}}} \mathbf{G}, \quad (14)$$

$$\frac{\partial\mathbf{c}_J}{\partial\mathbf{y}} = \frac{\partial\mathbf{c}_J}{\partial\mathbf{q}_{\text{sp}}} \mathbf{G}. \quad (15)$$

a) Subtlety in (15): The bias $\mathbf{c}_J$ depends on $\dot{\mathbf{q}}_{\text{sp}}$, which itself varies with $\mathbf{q}_{\text{sp}}$ through (5). Treating $\dot{\mathbf{q}}_{\text{sp}}$ as a free symbolic variable when differentiating $\mathbf{c}_J$ would miss the term $\partial\mathbf{G}/\partial q_i \dot{\mathbf{y}}$ for each spanning coordinate q_i , giving an incorrect Jacobian. The correct approach substitutes $\dot{\mathbf{q}}_{\text{sp}} = \mathbf{G}(\mathbf{q}_{\text{sp}}) \dot{\mathbf{y}}$ before differentiation, so that $\partial\mathbf{c}_J/\partial\mathbf{q}_{\text{sp}}$ is evaluated holding $\dot{\mathbf{y}}$ fixed. This correctly captures the full dependence of $\dot{\mathbf{q}}_{\text{sp}}$ on $\mathbf{q}_{\text{sp}}$.

D. Implementation via CasADi

`initializeDerivativeFunctions()` constructs three JIT-compiled CasADi functions at construction time. Symbolic variables $\mathbf{q}_{\text{sp}}^{\text{sx}}$, $\dot{\mathbf{y}}^{\text{sx}}$, $\mathbf{b}^{\text{sx}}$, and $\mathbf{F}^{\text{sx}}$ are declared, and $\mathbf{G}^{\text{sx}}$ is obtained by evaluating `Gfcn_` at $\mathbf{q}_{\text{sp}}^{\text{sx}}$ — returning a symbolic expression rather than a numeric value. The constrained spanning velocity is then $\dot{\mathbf{q}}_{\text{sp}}^{\text{sx}} = \mathbf{G}^{\text{sx}} \dot{\mathbf{y}}^{\text{sx}}$, which is used to drive all symbolic joint kinematics.

The three compiled functions are:

$$f_1 : (\mathbf{q}_{\text{sp}}, \mathbf{b}) \mapsto \frac{\partial(\mathbf{S}\mathbf{b})}{\partial \mathbf{q}_{\text{sp}}} \mathbf{G} \in \mathbb{R}^{6n \times n_v}, \quad (\text{dSb_dy_fcn})$$

$$f_2 : (\mathbf{q}_{\text{sp}}, \mathbf{F}) \mapsto \frac{\partial(\mathbf{S}^\top \mathbf{F})}{\partial \mathbf{q}_{\text{sp}}} \mathbf{G} \in \mathbb{R}^{n_v \times n_v}, \quad (\text{dSTF_dy_fcn})$$

$$f_3 : (\mathbf{q}_{\text{sp}}, \dot{\mathbf{y}}) \mapsto \frac{\partial \mathbf{c}_J}{\partial \mathbf{q}_{\text{sp}}} \mathbf{G} \in \mathbb{R}^{6n \times n_v}. \quad (\text{dSdotqd_dq_fcn})$$

Each function is compiled with common subexpression elimination (`cse = true`) and LLVM/Clang JIT (`-O3 -march=native`) to minimize evaluation overhead. At runtime, all three are evaluated via CasADi's low-level C API with pre-allocated input/output/work buffers (sized at construction via `sz_work()`), avoiding any per-call heap allocation in the control loop.